Volume 07 | Styling, Configuration, Deployment, and Legacy Boundaries

Understand CSS loading, runtime settings, dependencies and deployment.

1. CSS and templates

CSS files are loaded by templates; the cascade resolves by origin, importance, specificity, then source order. Shared theme variables provide colors/modes; dashboard polish files override earlier rules when selector strength ties. HTML IDs/class names are a contract with JS, so visual changes should not rename them casually. The landing and dashboard use distinct layout styles but shared theme code.

2. Runtime launch

`Procfile`: `web: uvicorn run:app --host 0.0.0.0 --port $PORT`. `run.py` direct execution uses host all interfaces, env PORT default 5000, optional reload flag. Uvicorn imports `run:app` under Procfile. FastAPI docs are under `/api/docs`. Configure Render build to install `requirements.txt`; start command should be consistent with Procfile.

3. Declared dependencies

FastAPI, Uvicorn standard extras, python-multipart (form parser), Jinja2, joblib, NumPy, Pandas, python-dateutil, scikit-learn, SciPy. Requirements ranges/versions describe permitted installs, not proof of runtime installed versions. Reproducible deployments benefit from a lockfile and tested compatible model artifact version. Flask/Gunicorn are no longer declared runtime dependencies in this migration snapshot.

4. Files loaded at runtime vs training-only data

Runtime needs serialized estimator and metadata, cleaned stations, operations CSV, topology Python module, optional line-distribution JSON, templates/static files, calendar and hours code. The 3.36M-row synthetic training CSV is consumed by training pipeline, not loaded by each regular prediction request. If Render rebuilds/retrains, that CSV or a data-generation step is needed. The model artifact must be shipped through a reliable trusted artifact path; not including the large CSV does not make serving impossible if model artifacts and runtime inputs are present.

5. Import-time coupling and workers

Routes read station data/build graph on import; forecasting/scheduler load files on import. Startup failures can happen before health check. Each worker process loads its own objects, multiplying memory. A lifespan handler could explicitly load and validate models/data; do not move model training into startup.

6. Legacy boundary

This archive is an older Flask-era documentation set. Current app code uses FastAPI decorators, Request, Jinja2 TemplateResponse, JSONResponse/HTMLResponse, StaticFiles and Uvicorn. Do not copy old examples that import Flask, use `flask.request`, `render_template`, or Gunicorn WSGI commands. Browser semantics remain mostly independent of backend framework, but route return/error contracts may differ.

7. Deployment verification caveat

Previously checked Python syntax and diff hygiene; the environment lacked FastAPI packages and network access prevented installing them, so runtime smoke was not possible in that environment. Before claiming the migration runs in production, install requirements in a clean env, run Uvicorn, test landing/dashboard, `/api/stations`, `/api/model_metrics`, valid and invalid `/api/predict`, form route, static assets, and closed time. Then inspect Render deploy logs and health response.